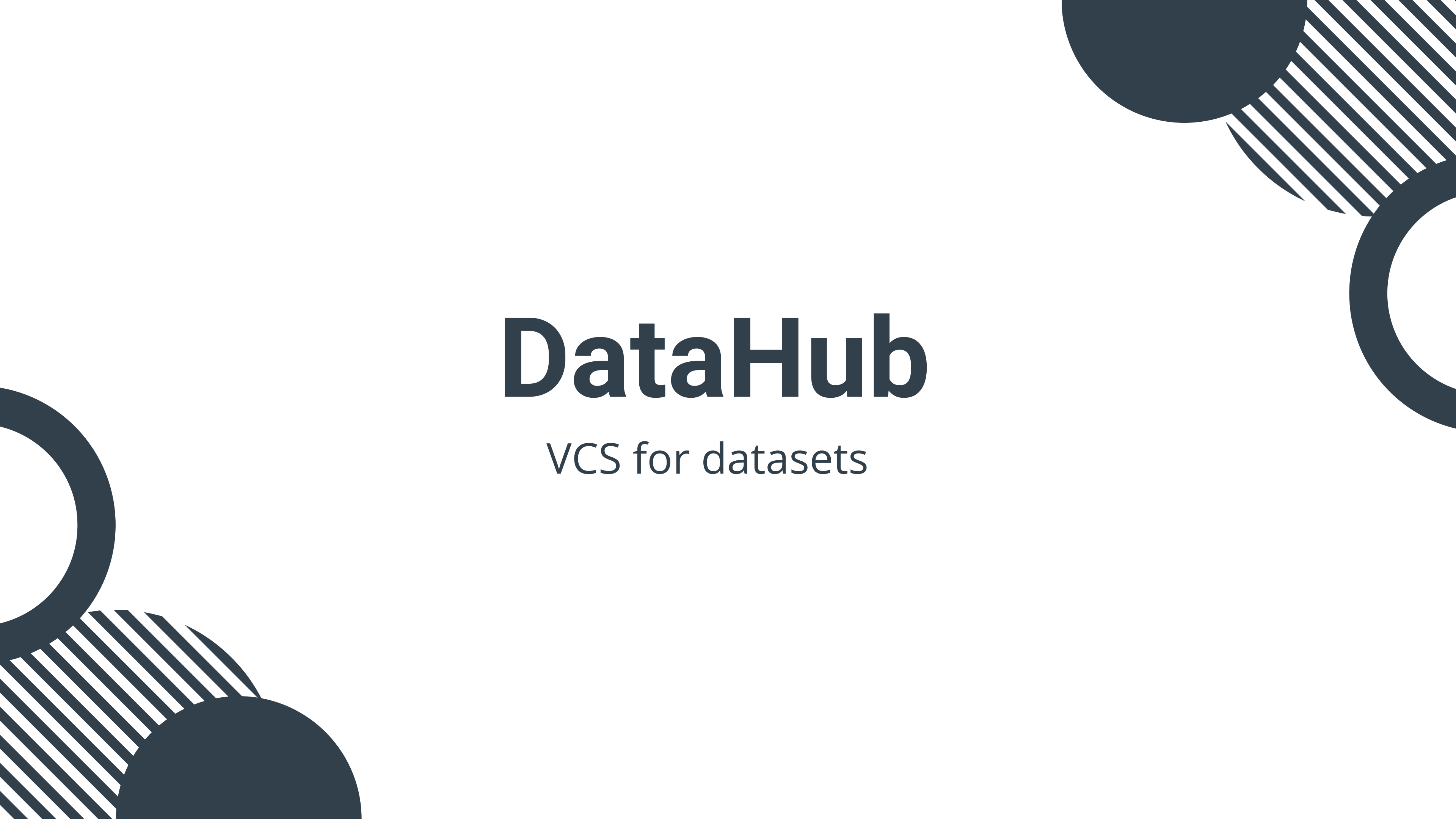
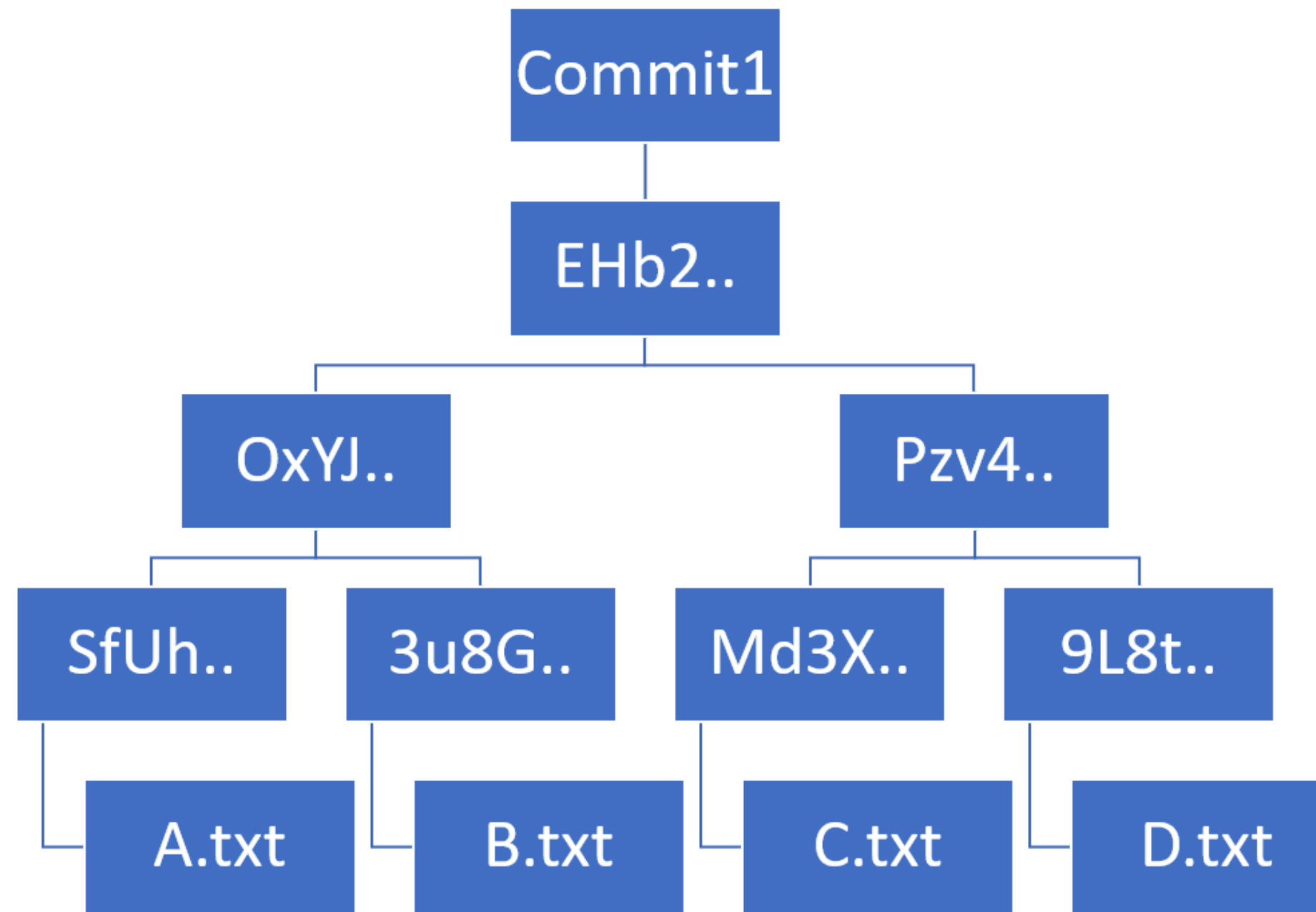


The image features a white background with decorative elements in the corners. In the top-right and bottom-left corners, there are overlapping circles. Some circles are solid dark blue, while others are filled with a pattern of dark blue diagonal stripes. The text is centered in the middle of the page.

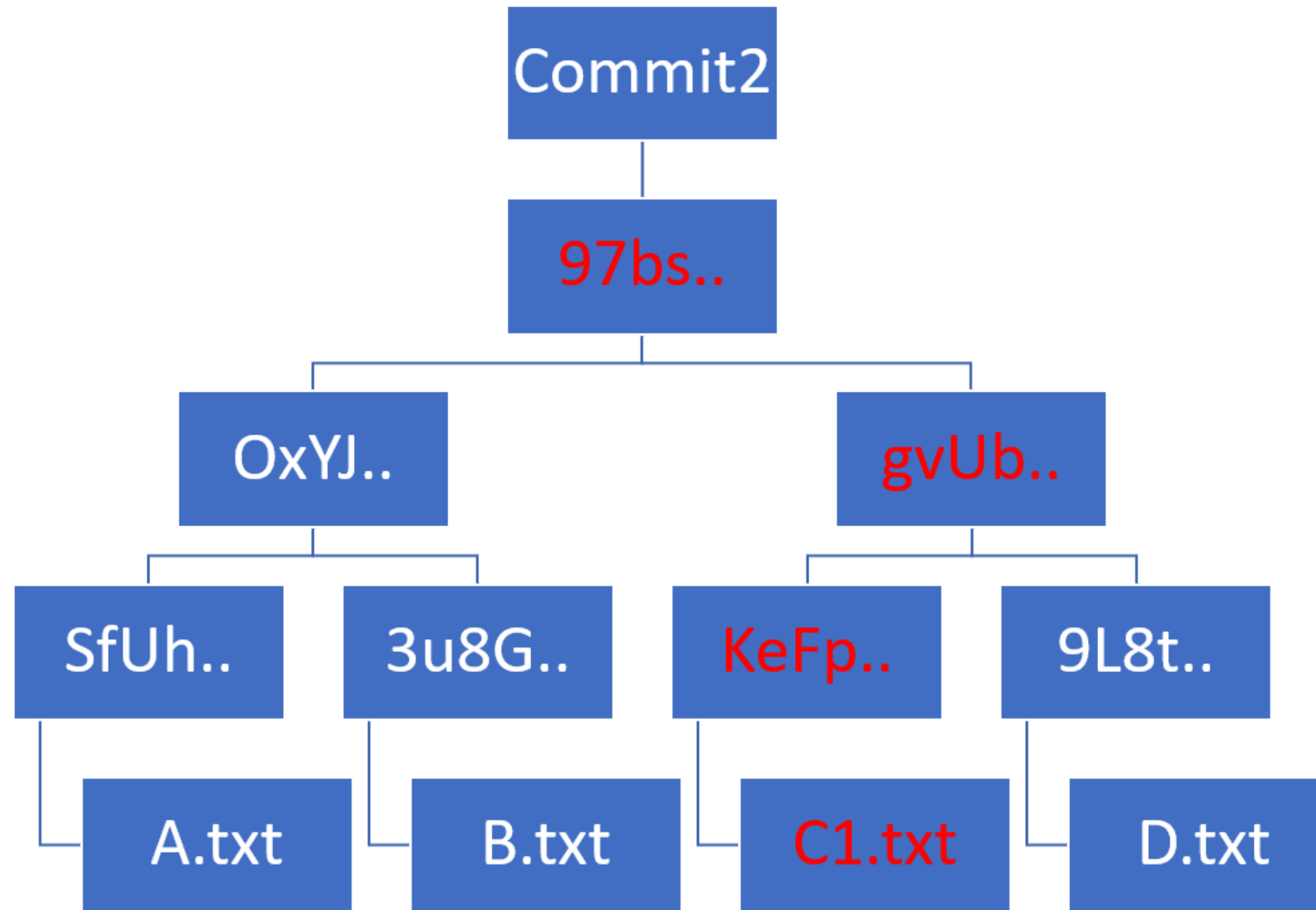
DataHub

VCS for datasets

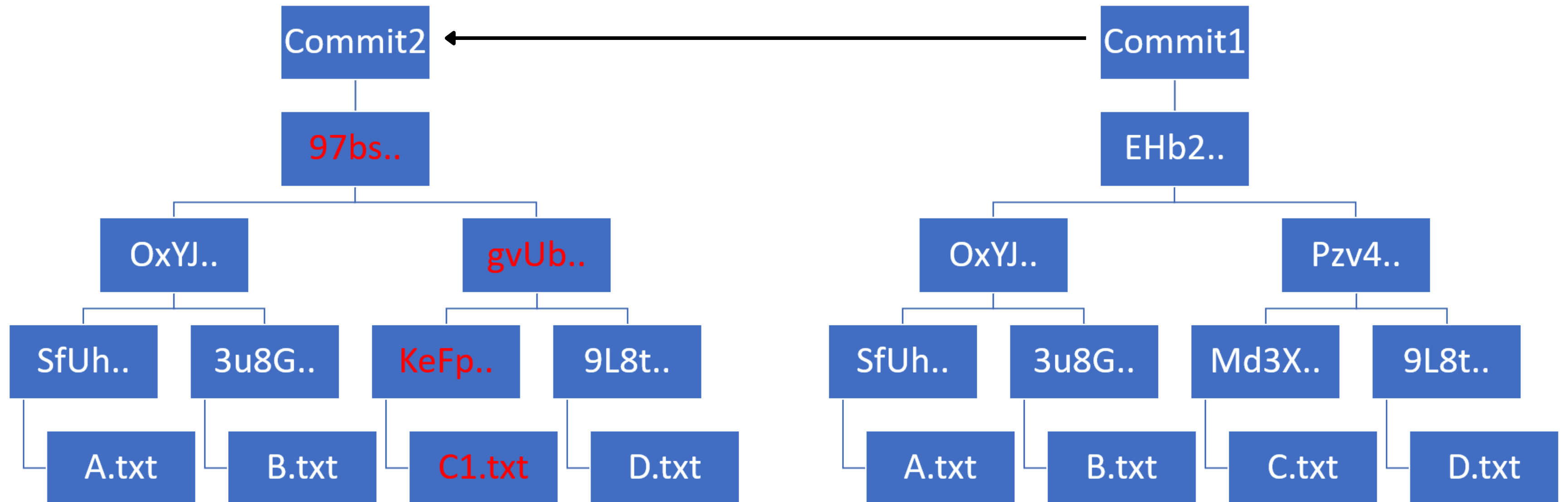
What is Merkle Tree ?



Changed C to C1



Connecting the two trees / commits



Content Addressable Storage v/s Traditional Storage

Traditional Storage

LOCATION-BASED

FILE ADDRESS

`/var/datasets/v1/data.csv`

Location Dependent: Data retrieved by folder directory path.

Silent Data Drift: File contents can alter while keeping same path.

Duplicate Overhead: Identical files stored across multiple locations.

File Location	BLOB_Addr
/desktop/report.pdf	#Gzg9
/desktop/report1.pdf	#hsf0

Even though the content is same, the system treats them as 2 seperate files

Content Addressable Storage v/s Traditional Storage

File Hash	BLOB_Addr
HASH_XYZ	#Gzg9
HASH_ABC	#hsf0
HASH_XYZ	#Gzg9

The key difference is that the content itself determines the BLOB address, via a hash.

Content-Addressable (CAS)

HASH-BASED

FILE ADDRESS (SHA-256)
a1b2c3d4e5f6 ...

Cryptographic Identity: Data address is computed directly from raw content.

Self-Verifying: Hash acts as address and instant integrity proof.

Native Deduplication: Identical content yields identical address.

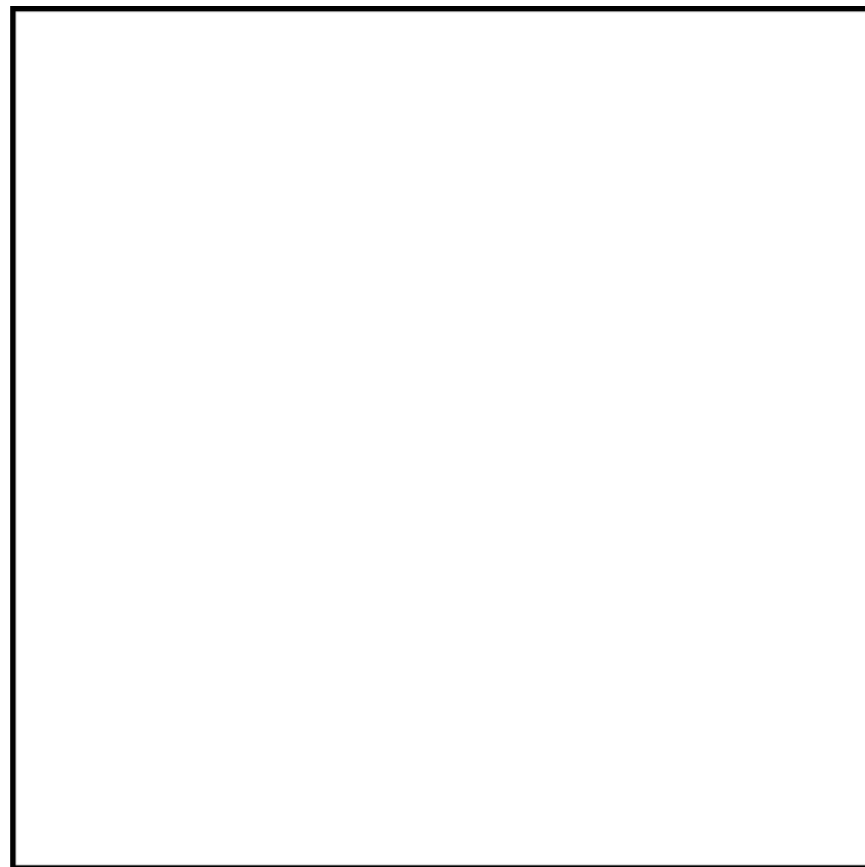
Problem with traditional Merkle Tree

Hash	Content
SfUh	A
3u8G	B
Md3X	C
9L8t	D
KeFp	C1

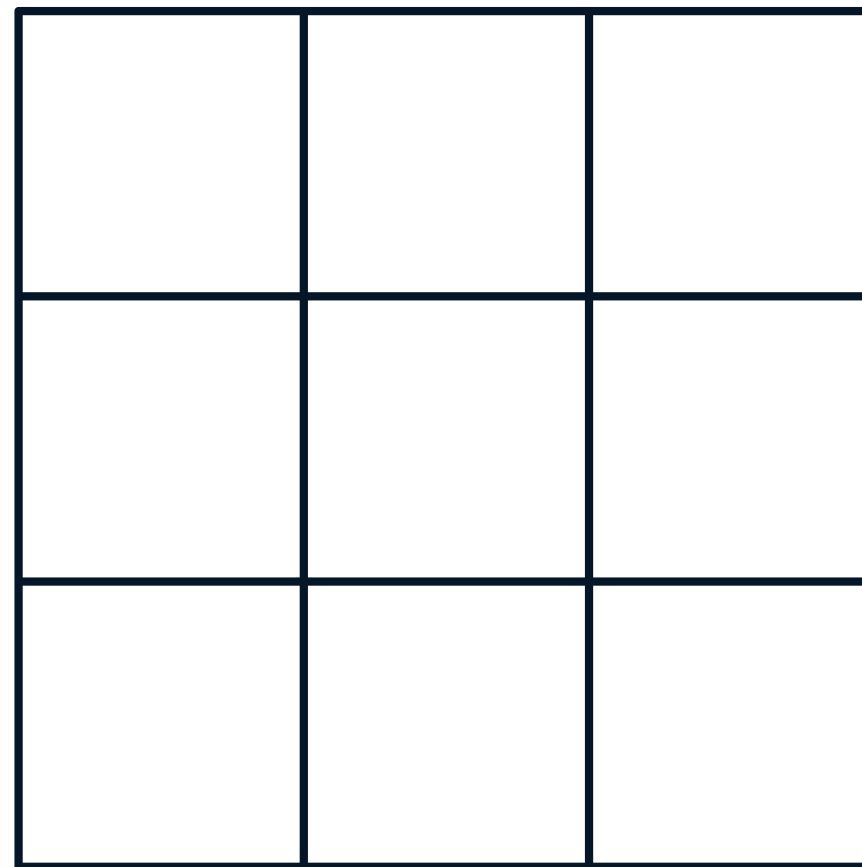


C1 is still stored as a separate file

Solution : Custom datatype



Assume this is C.txt

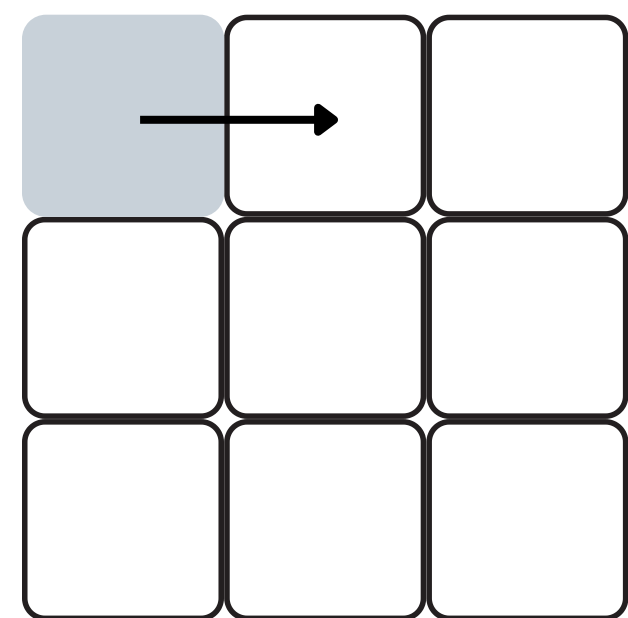


we divide it into chunks of 16
MB each



Instead of the entire file, we work
with these chunks

How it works ?

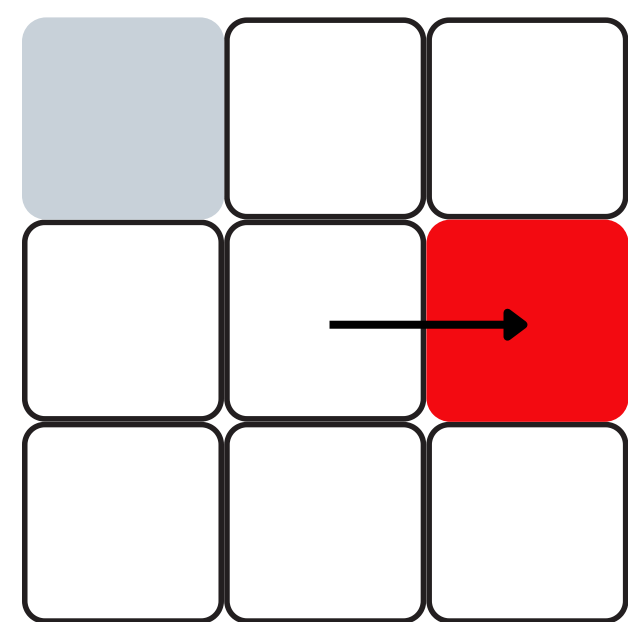


Original C.txt

Chunk_No	Chunk_Hash	BLOB_Addr
Chunk_1	rG8n..	#B8kL
Chunk_2	9qYM..	#Bgf7
Chunk_3	u4FT..	#O9kC
.....	...	...
Chunk_N	ac54..	#Hy0z

Now, instead of the file as the leaf nodes, we store this index table in the merkle tree

How it works ?

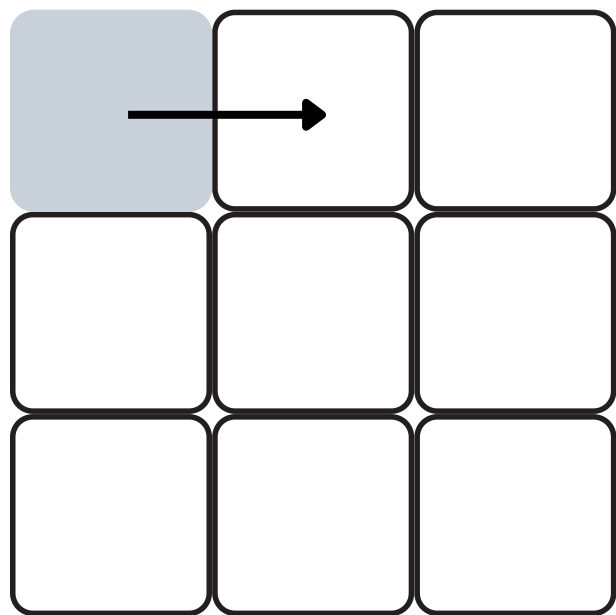


modified C1.txt

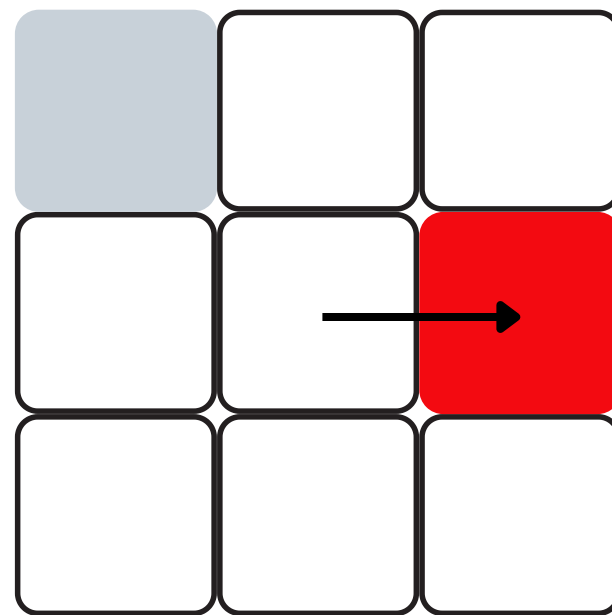
In our master storage, we only store the new chunk

Chunk_No	Chunk_Hash	BLOB_Addr
Chunk_1	rG8n..	#B8kL
Chunk_2	9qYM..	#Bgf7
Chunk_3	u4FT..	#O9kC
Chunk_6	bvrE..	#tbf0
....	...	...
Chunk_N	ac54..	#Hy0z

How it works ?



Original C.txt



modified C1.txt

This allows us to store only the diff rather than the entire file

Assume a dataset that is ~10GB

$v1.csv = 10GB + v2.csv = 10GB$

Total 20GB would be used

Now instead, only a few extra MB is used
So instead of 20GB, we'll be requiring only 10.8 GB (example)